



CS61C

Great Ideas
in
Computer Architecture
(a.k.a. Machine Structures)



Assistant
Teaching Professor
Lisa Yan

Summary, What's Next?



End of Semester Details

- End of Semester Details
- Great Ideas in Computer Architecture
- Courses after CS61C
- Beyond Coursework
- Conclusion



End of Semester

<https://edstem.org/us/courses/93340/discussion/7994974>

<https://cs61c.org/sp26/exam/>

	Sun	Mon	Tue	Wed	Thur	Fri	Sat
Week 14 (classes)					HW 10		
RRR week	Prof. Yan's OH: on calendar Review Session: Th 5/7 10am-2pm (with 30min break)						
Finals Week			<u>Final Exam</u> 7-10pm PT				



Final Exam Details

<https://edstem.org/us/courses/93340/discussion/7994974>

<https://cs61c.org/sp26/exam/> ←←←

- Cumulative final
- 3 hour exam, Tuesday, May 14th, 7:00 PM - 10:00 PM PT
- 2 **handwritten** double-sided sheets of paper allowed
- We will provide the full 61C reference sheet
- Z-score clobber (if you do better on the final)
- Review session recordings + past exam materials released soon
- RRR week: Big review session/OH Th 5/7 10am in Woz
 - Professor Yan OH: M (advising) and E (conceptual), on calendar soon



Agenda

Great Ideas in Computer Architecture

- End of Semester Details
- Great Ideas in Computer Architecture
- Courses after CS61C
- Beyond Coursework
- Conclusion

We made HW/SW Contact!!!



High Level Language
Program (e.g., C)

Compiler

Assembly Language
Program (e.g., RISC-V)

Assembler

Machine Language
Program (RISC-V)

Hardware Architecture Description
(e.g., block diagrams)

Architecture Implementation

Logic Circuit Description
(Circuit Schematic Diagrams)



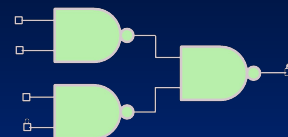
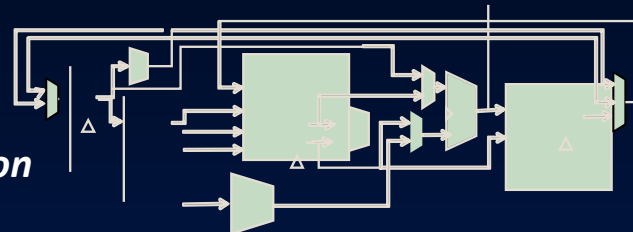
The Creation of Adam,
Michelangelo, c. 1512

```
temp = v[k];
v[k] = v[k+1];
v[k+1] = temp;
```

```
lw    x3 0(x10)
lw    x4 4(x10)
sw    x4 0(x10)
sw    x3 4(x10)
```

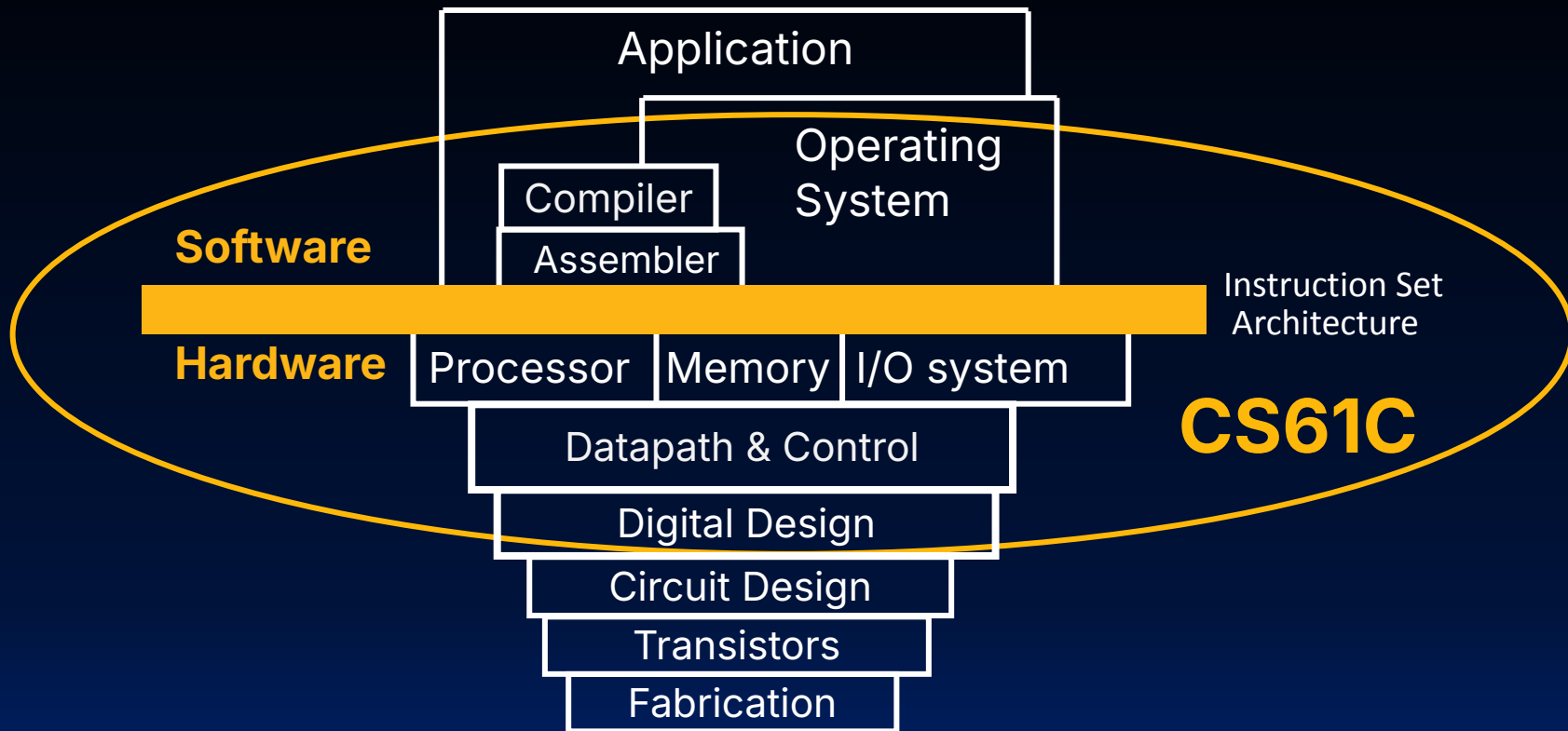
```
1000 1101 1110 0010 0000 0000 0000 0000
1000 1110 0001 0000 0000 0000 0000 0100
1010 1110 0001 0010 0000 0000 0000 0000
1010 1101 1110 0010 0000 0000 0000 0100
```

Anything can be a
number—data,
instructions, etc.





We learned Old School “Machine Structures”...



New school Machine Structures is much more complicated!

Software

Parallel Requests

Assigned to computer
e.g., Search "Cats"

Parallel Threads

Assigned to core
e.g., Lookup, Ads

Parallel Instructions

>1 instruction @ one time,
e.g., 5 pipelined instructions

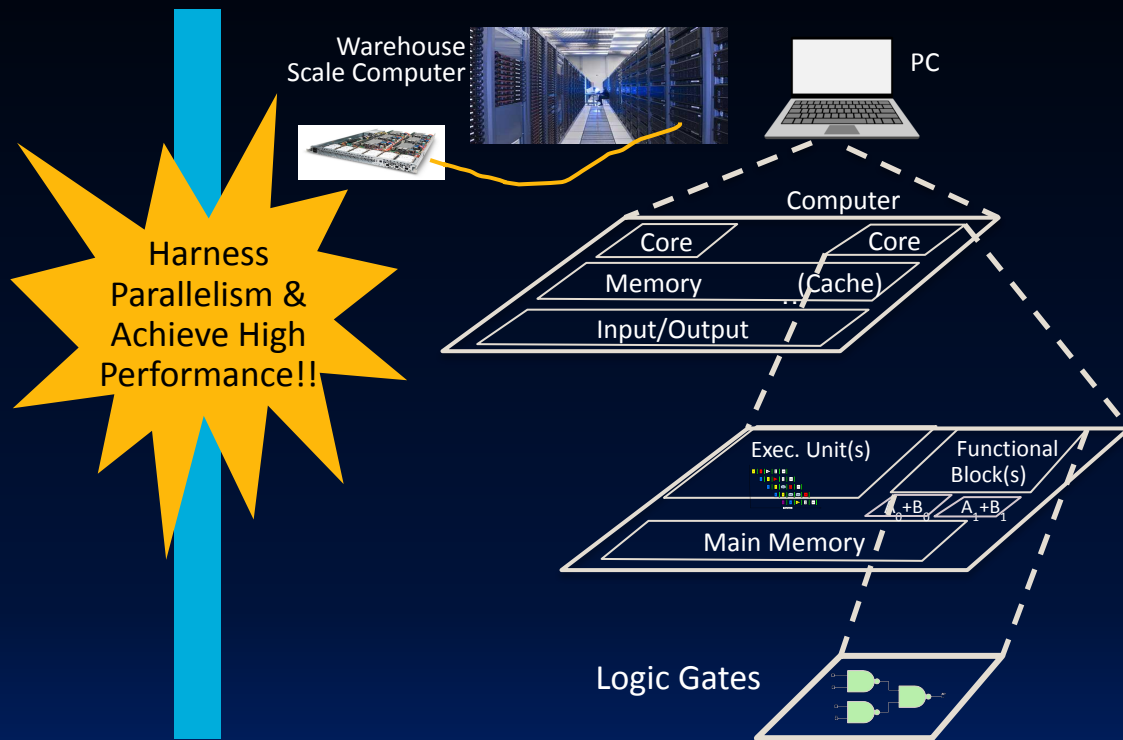
Parallel Data

>1 data item @ one time
e.g., Add of 4 pairs of words

Hardware descriptions

All gates work in parallel, at same time

Hardware





6 Great Ideas in Computer Architecture

1. Abstraction (Layers of Representation/Interpretation)
2. Moore's Law
3. Principle of Locality/Memory Hierarchy
4. Parallelism
5. Performance Measurement & Improvement
6. Dependability via Redundancy



Courses after CS 61C

- End of Semester Details
- Great Ideas in Computer Architecture
- Courses after CS61C
- Beyond Coursework
- Conclusion



Prof Yan's EECS path

- Sophomore Fall (Semester after Summer CS61C)
 - CS188: Artificial Intelligence
 - ^ first upper div!
 - Math 54/Math 55
 - Eng 45: Props of Materials
 - Hist 127AC: History of California
 - Research Position 1
- Most of my time:
 - Tennis at Cal, ESC, HKN tutoring
 - ~~Late night boba runs~~
- Junior Fall
 - EE 120: Signals & Systems
 - EE 126: Probability & Random Processes
 - Physics 137A: Quantum Mechanics
 - Japan 188: Visual Culture in Anime
 - Research Position 2



Prof Yan's EECS path

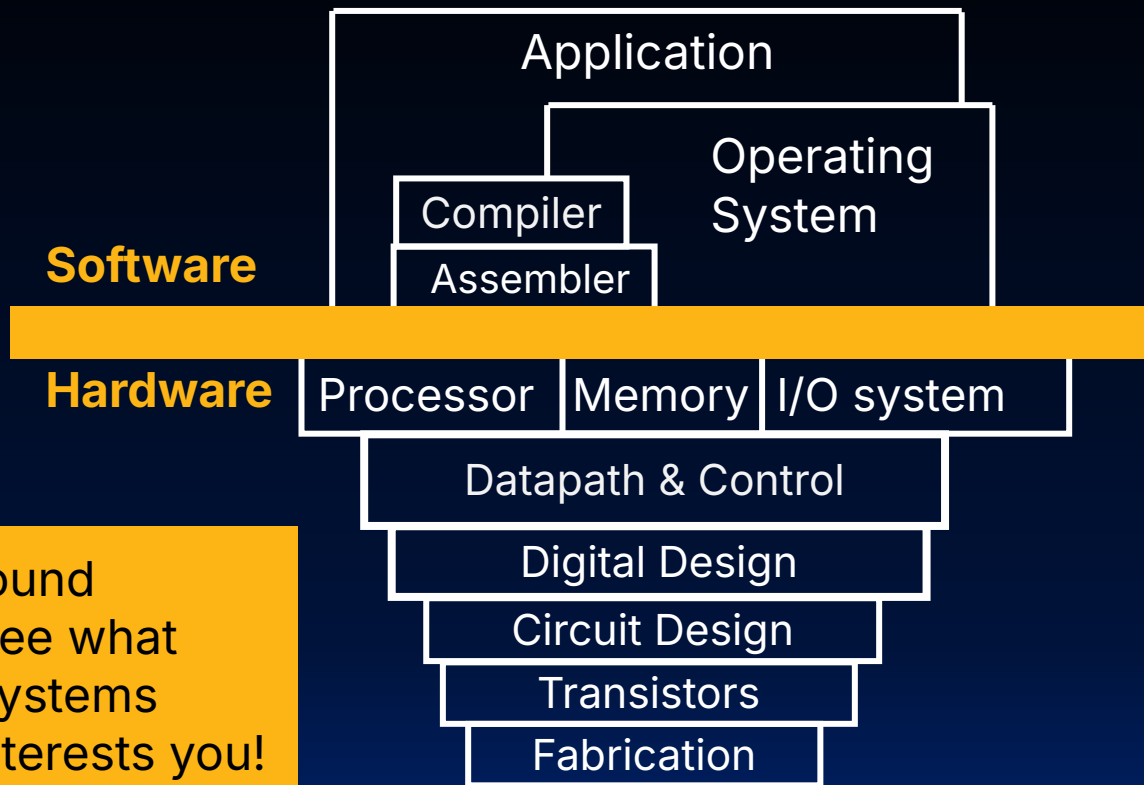
- Junior Spring
 - Computer Networks
 - Omg I love this!
 - Wireless Comms (Grad)
- Senior Fall
 - EE 20N TA (signals and systems)
- Senior Spring
 - EE 20N Head TA
 - 12 unit P/NP
- Most of my time:
 - Tennis at Cal, Cal Day/ESC, HKN tutoring
 - ~~Late night boba runs~~

- Stanford MS/PhD
 - "All things systems"
 - Networks:
 - Software-defined networks
 - Domain-specific programming language, P4
 - Grad/ugrad networks education
 - Teach intro CS at scale, but with humans
 - Thesis: Tools to Understand How Students Learn
 - Three years of East Asian language courses



Some courses in the SW/HW stack

<https://edstem.org/us/courses/93340/discussion/7890514>



- CS 168 (networks)
- CS 189 (ML)
- CS 164 (compilers, PL)
- CS 162 (OS)
- CS 152 (architecture)
- EECS 151 (design)
- EE 144 (verification)
- EE 105 (circuits)
- EE 113 (power)



Agenda

Beyond Coursework

- End of Semester Details
- Great Ideas in Computer Architecture
- Courses after CS61C
- Beyond Coursework
- Conclusion



The best way to learn is to teach!

Join course staff?

- <https://eecs.berkeley.edu/resources/gsis/prospective>
- Fall 2026 applications open...!!!
 - UCS1 (tutor/reader combo appt, office hours and grading)
 - TA/UCS2/GSI (section leader)
 - [Financial Aid FAQ](#)

We strongly encourage you to follow this path!

- Build your confidence in technical depth
- Improve your professional communication skills
- Empower others and improve the climate and belonging in EECS
- Join a super fun community of lifelong learners and teachers



Cal Opportunities: Research (1/2)

Why are we one of the top universities in the WORLD?

- Research, research, research!
- More importantly: We develop **independent thinkers** who can **communicate their ideas to others**.

Whether you go into academic or industry, you need someone to vouch for you.



Cal Opportunities: Research (2/2) in Sys/Arch

Why are we one of the top universities in the WORLD?

- Research, research, research!
- More importantly: We develop **independent thinkers** who can **communicate their ideas to others**.
- From general-purpose applications:
 - Make Python run like C with good compilers and hardware
 - Déjà vu: Just like making higher-level languages fast on RISC
- To Domain-specific applications (bigger opportunities!)
 - What are the right DSLs for important applications?
 - Co-design of new DSLs and DSAs
 - Advanced compilation techniques for optimizing the matching!



Research Labs!

<https://sky.cs.berkeley.edu/>
<https://slice.eecs.berkeley.edu/>



Joseph Gonzalez

Core Faculty



Raluca Ada Popa

Core Faculty



Ion Stoica

Core Faculty



Koushik Sen

Core Faculty



Natacha Crooks

Core Faculty



Matei Zaharia

Core Faculty

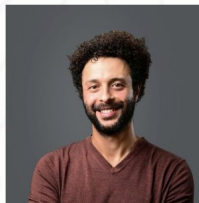


Sky
Computing
Towards
Utility
Computing
for the Cloud



Krste Asanović

Faculty



Aydin Buluç

Faculty



Alvin Cheung

Faculty



James Demmel

Faculty



Sagar Karandikar

Faculty



Jelani Nelson

Faculty



Bora Nikolić

Faculty



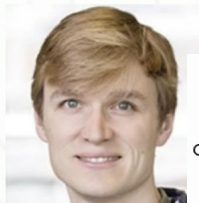
Sophia Shao

Faculty



Kathy Yelick

Faculty



Christopher Fletcher

Faculty

The SLICE Lab at UC Berkeley aims to democratize the design, programming, and integration of domain-specific computing at scale so that a wide-range of applications can benefit from specialized hardware, extending the continued growth in computational capabilities for future-generation computing ecosystems.



More Research Labs!

<https://epic.berkeley.edu/>

<https://bwrc.berkeley.edu/people/directors>



Sarah Chasins
Faculty Co-Director ☆



Joe Hellerstein
Faculty Co-Director ☆



Aditya Parameswaran
Faculty Co-Director ☆



Alvin Cheung
Faculty



Joseph Gonzalez
Faculty



Björn Hartmann
Faculty



Marti Hearst
Faculty



Anthony Joseph
Faculty



Erin Kerrison
Faculty



Michael Mahoney
Faculty



Niloufar Salehi
Faculty



Koushik Sen
Faculty



Dawn Song
Faculty



Matei Zaharia
Faculty



BWRC

Berkeley Wireless Research Center

EPIC DATA lab

The mission of the EPIC Data lab (short for **E**ffective **P**rogramming, **I**nteraction, and **C**omputation with Data) is to develop *low-code* and *no-code* interfaces for data work, powered by next-generation predictive programming techniques. The EPIC Data lab follows in the long



Ana Arias
Associate Professor
Electrical Engineering and Computer Sciences



Jun-Chau Chien
Assistant Professor
Electrical Engineering and Computer Sciences



Ricky Muller
Assistant Professor



Ali Niknejad



Borivoje Nikolic



Jan Rabaey



Sophia Shao



John Wawrzyniec

Yan, SP26





Search EECS by Area

<https://www2.eecs.berkeley.edu/Research/Areas/>

- Professor titles: Assistant, Associate, Full
- Professor types: Research, Teaching
- Where to find grad students
- Where to find undergrad researchers



How do I get into research?

Cold emails can work, but most opportunities are built through investing in the interpersonal (cf Dave Patterson)

- DARE: <https://dare.berkeley.edu/>
- URAP: <https://research.berkeley.edu/urap/>
 - Fall apps open late August
- SUPERB:
<https://eecs.berkeley.edu/resources/undergrads/research-2/superb/>
 - Summer apps open around November
- BeeHive:
<https://engineering.berkeley.edu/research-and-faculty/undergrad-research/>
- Mailing Lists!!!
- **Events!!! Public talks!!!** <https://events.berkeley.edu/eecs/all>
 - *It's okay not to know!!*
Lots of cool dissertation talks and guest speakers
- Cold emails! **Graduate students!!!**



Lisa's Research and Development Projects

Computing/Data Science Education

- Sociotechnical Systems
 - CS 195: Society Impacts
Technology Impacts Society
- Systems education
 - Data 101: Data Engineering
 - Data 6: Computational Social Science + CS + Data
 - CS 61C: architecture

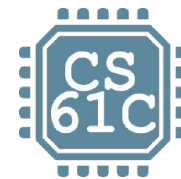
Higher Ed Teaching Infrastructure

- Improving the TA experience in EECS/Data
 - Pedagogy courses
 - Teach teachers how to teach
- Seamless Learning
 - Make custom course tools accessible, secure, and open-source



CS 61C Industry & Research Panel

May 5 (Tue), 2-3pm, Wozniak Lounge; bit.ly/61c-panel

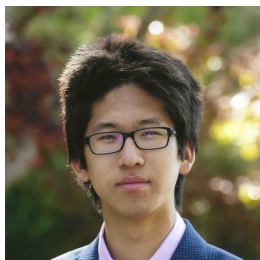


🍕 Complementary pizza will be provided 🍕



Nicolas Rakela

- BS EECS'26 - UCB
- MS EECS'27 - UCB
- SLICE/BWRC Lab
- EECS 16A, EECS 16B, EECS 151, Tapeout TA
- Apple Intern ('25, '26)
- Apple MS Scholar



Jim Fang

- BS EECS'27 - UCB
- SLICE/BWRC Lab
- Former CS 61C TA
- Tapeout Head TA
- IEEE Chair
- Google Student Researcher ('25-'26)



Professor Sagar Karandikar

- BS EECS'15 - UCB
- PhD EECS'24 - UCB
- SLICE/BWRC Lab
- Former 6x CS 61C TA (4x as Head TA) and Lecturer
- Google Student/Faculty Researcher ('20-'25)



Professor Chris Fletcher

- BS EECS'10 - UCB
- PhD EECS'16 - MIT
- SLICE/BWRC Lab
- Former UIUC Professor
- NVIDIA Architecture Research Group ('16-'17)



Andre Green

- BS EE'23 - Penn
- MS EECS'25 - UCB
- PhD EECS'28 - UCB
- SLICE/BWRC Lab
- Former CS61C TA
- IBM Intern ('22-'23)
- NVIDIA Architecture Research Group ('25-)



Kelly Tou

- BS EECS'27 - UCB
- Former SWARM Lab
- SLICE/BWRC Lab
- EECS 151 TA
- 2x Apple NSI Design Winner
- Apple Intern ('25, '26)



Conclusion

- End of Semester Details
- Great Ideas in Computer Architecture
- Courses after CS61C
- Beyond Coursework
- Conclusion

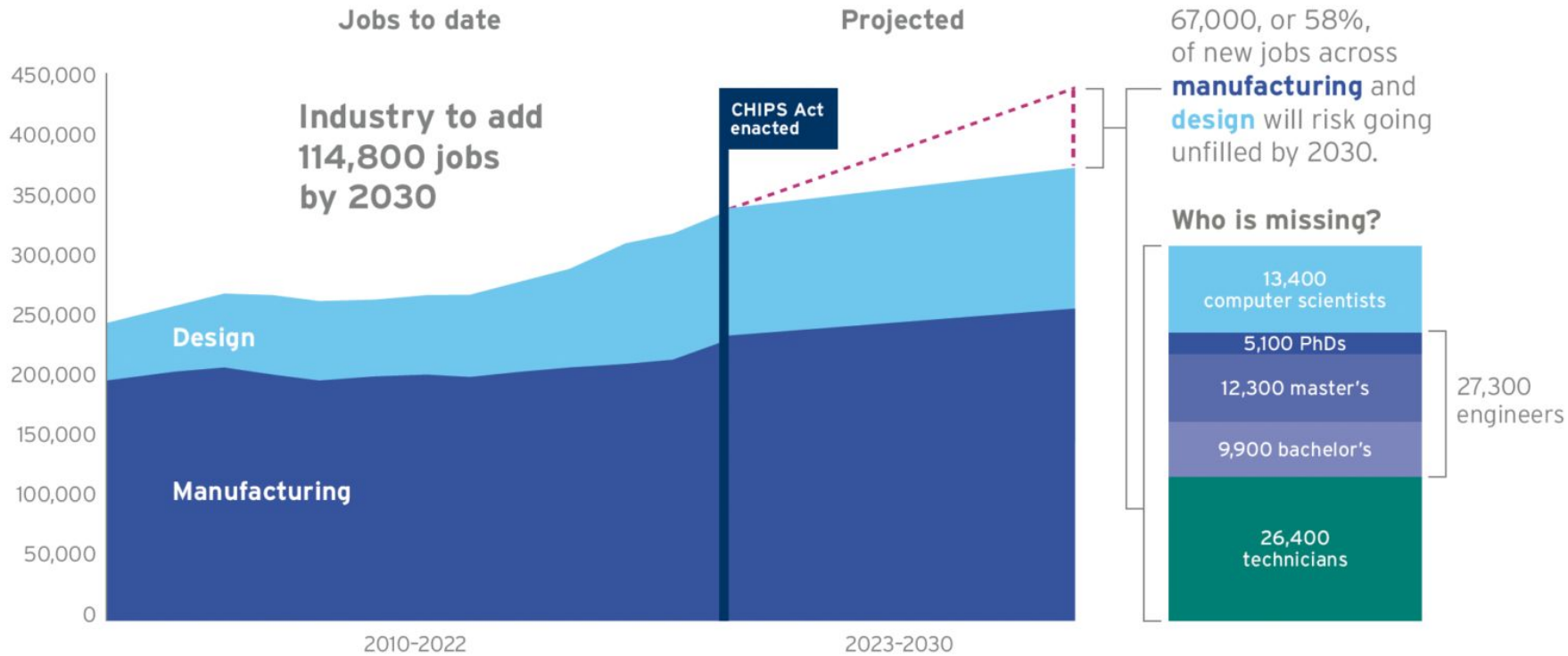


What's a future in EECS? (1/2)

- Honestly? It's hard to say.
 - The **technical material** is more interesting than ever.
 - Compute power, via the **cloud**, is cheaper than ever.
 - There is a renewed national interest in the **semiconductor industry**.
 - Dave Patterson: <https://www.shapingai.com/>
 - **The fundamentals that you learn at UC Berkeley are more important than ever.**

Semiconductor workforce: projected gap

FIG. 1: Historical semiconductor workforce and projected 2023-2030 gap





What's a future in EECS? (2/2)

- But computing has never before been so **undemocratic**:
 - The forefront of innovation is often in **for-profit companies**.
 - Corporate and government surveillance are sustained by late-capitalist incentive structures that concentrate wealth.
 - The Bay Area has the **deepest, widest income inequality** nationwide.
 - **Our students often can't get jobs**, much less get into courses.
 - **Our public education and research are systematically underfunded**.
 - Local, national, and global systems are suppressing, oppressing, and incarcerating voices of the minoritized and of the innovative.
 - AI is accelerating a widening gap in an already broken education and research system.
- These systems and patterns prevent access to the **education and opportunities** that everyone deserves.



So what do we do?

Enter the 7th Great Idea of
Computer Architecture.



6 Great Ideas in Computer Architecture

1. Abstraction (Layers of Representation/Interpretation)
2. Moore's Law
3. Principle of Locality/Memory Hierarchy
4. Parallelism
5. Performance Measurement & Improvement
6. Dependability via Redundancy

Yan, SP26



7. We stand on the shoulders of giants

- All the technologies, all the paradigms we taught you in this class were **designed by people**.

def **politics**, n. Decision-making in groups to negotiate power, resources, and status.

Engineering is inherently **political**.
Engineering is **human and messy!**

7. We stand on the shoulders of giants

- All the technologies, all the paradigms we taught you in this class were **designed by people**.
 - Why don't caches and VMs have the same terminology?
 - Who standardized 32-bit? 64-bit?
 - Who wrote IEEE 754 (Single Precision Floating Point)?
 - Who made RISC-V the ISA of the present and future?

Prof Dave Patterson and others, UC Berkeley, ***and RISC-V Intl, an association with industry and academic partners that impact the global development of technology and policy.***

Will Kahan, Prof. Emeritus, UC Berkeley, ***with a team of scientists/researchers at Intel and IEEE (Institute of Electrical and Electronics Engineers).***



Look around you.



*You are surrounded by your
classmates at **the top public
institution in the world...***

*...but more importantly, you are surrounded by
your future teammates, mentees, managers,
policy makers, humanists, professors, advisors,
friends, family, heroes, and **biggest supporters.***



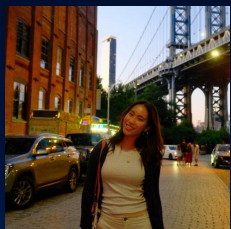
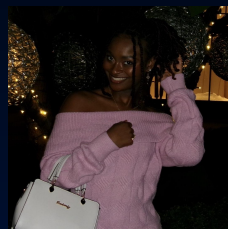
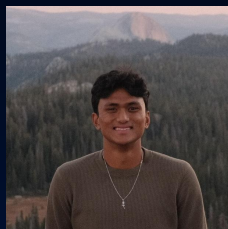
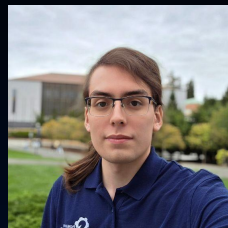
*You are surrounded by your
classmates at **the top public
institution in the world...***

*...but more importantly, you are surrounded by
your future teammates, mentees, managers,
policy makers, humanists, professors, advisors,
friends, family, heroes, and **biggest supporters.***

*So please, remember **your humanity.**
Our persistent, unrelenting humanity is
what makes us **great engineers.***



Great Idea #7 in Action!



Thank you to ALL the course staff!!!

Thank you to all the former CS61C instructors who have built out this course!!!

UC Berkeley

40-Summary, What's Next? (34)





...and finally...

Thank YOU!!!

And thank you in advance for filling out course evaluations!



Agenda

Courses after CS 61C, cont.

- End of Semester Details
- Great Ideas in Computer Architecture
- Courses after CS61C
- Beyond Coursework
- Conclusion



CS 162: Operating Systems

- Primary focus: Hardware, Software
- Expands on the concepts of the OS and VM
- Uses C a lot, so it helps to be comfortable with explicit memory management
- There's also a lot of **multiprocess programming**.
 - Locks, deadlocks, timers, etc.
- The main project sequence goes through writing a simple OS, and adding various bits of functionality to it.



CS 161: Computer Security

<https://edstem.org/us/courses/73598/discussion/6469238>

- Primary focus: **Hardware, Software, Theory**
- Part 1: How to hack programs, and how to prevent your programs from being hacked
- Part 2: How the internet works (because the internet is terribly insecure)
- Ends up affecting **all layers of the tech stack**, so it's fairly fast-paced
- Very important to take:
 - Good first upper div! But it's also not urgent.



CS 164: Compilers and Programming Languages

<https://edstem.org/us/courses/73598/discussion/6469238>

- Primary focus: Software, Theory
- The "C" part of **CALL**
- Has a lot of connections to linguistics and parsing
- The main project sequence goes through writing a compiler from (a variant of) Python to RISC-V, step by step
 - Lexer: Translate code into sequences of tokens (letters→words)
 - Parser: Translate tokens to meaningful statements (words→sentences)
 - Code Generation: Translate meaningful statements into the new language (sentences→new language)



EECS 151: Intro to Digital Design

<https://edstem.org/us/courses/73598/discussion/6469238>

- Lab-based class: FPGA or ASIC (or both!)
 - **FPGA**: Field Programmable Gate Array, will be hands-on based (working with an FPGA board)
 - **ASIC**: Application-Specific Integrated Circuit, will be simulated
 - Both different ways in building and designing chips to be shipped for production!
- Focuses on datapath/SDS sections of course
 - A lot of things we've abstracted away in 61C become clearer!
 - Delays of hardware components
 - Various gates and how they're created
 - FSMs
 - More datapath
- Introduces Verilog



- **Branch prediction**—remember that?
 - It's an entire research field, and there are many different methods by which we can improve execution!
 - Bitwise manipulation + tracking, mostly
- One-instruction executed per cycle max (even during pipelining)
 - No longer the case with instruction-level parallelism!
 - ILP: multiple instructions get executed per clock cycle!
 - Architecture that supports ILP include
 - Superscalar architecture
 - VLIW: very long instruction words (152)
- Vectorised architecture, SIMD which is Intel-supported
 - What about RISC-V vectorised architecture?
 - Currently undergoing research at Berkeley!
 - BAR: Berkeley Architecture Research
 - (Vectorising radix-based sorting algorithms?)



EE 194/290: Tapeout

<https://edstem.org/us/courses/73598/discussion/6469238>

- *"In 15 weeks we will design a single-chip wireless sensor node "System on a Chip" with a 32 bit RISC-V processor and memory, 2.4 GHz radio transceiver, machine learning accelerator, energy scavenging interface, power management, on-board sensors, and sensor interfaces."*
- "The (extremely challenging) goal is to send the chip out for fabrication in 16 nm CMOS at the end of the semester."
- "One or more teams of students will divide the project into modules and work together to design and integrate the modules into a single chip."
- Prerequisite: **EE 151**
- Spring 2025:

<https://classes.berkeley.edu/content/2025-spring-eleng-198-020-grp-020>



EE 194/290: Bringup

<https://edstem.org/us/courses/73598/discussion/6469238>

- Test Integrated Circuit Chips Designed in Previous Tapeout Classes
- Build printed circuit boards and write software to enable testing and debug of the chips. Create bug fixes, and specs for the next generation of chips.
- Pre-requisites: Tapeout class the previous semester, or plans to take the tapeout class the following semester.
- Fall 2025:
<https://classes.berkeley.edu/content/2025-fall-eleng-194-001-lec-001>



200-level classes

<https://edstem.org/us/courses/73598/discussion/6469238>

- Many graduate-level classes also allow undergraduates to enroll with permission!
- Often go deeper into concepts covered in upper-division classes
 - CS 267: Parallel computing
 - CS 263: Compilers
 - CS 294: Various topics (based on the semester)
- **Many of these classes are rarely taught**, so take them when you can!
- Primary difference from undergraduate classes: Most graduate courses have no final, but instead have a final project, where you design or analyze an emerging research topic.